目录

OpenFR 2
摘要 2
第一部分　作品概述 2
1.1 功能与特性 2
1.2 应用领域 3
1.3 主要技术特点 4
1.4 主要性能指标 4
1.5 主要创新点 5
1.6 设计流程 5
第二部分　系统组成及功能说明 5
2.1 整体介绍 5
2.2 硬件系统介绍 8
2.2.1 硬件整体介绍 8
2.2.2 电路各模块介绍 8
2.3 软件系统介绍 14
2.3.1 软件整体介绍 14
2.3.2 软件各模块介绍 15
第三部分　完成情况及性能参数 16
3.1 整体介绍 16
3.2 工程成果 18
3.2.1 硬件成果 18
3.2.2 软件成果 19
3.2.3 附加功能一 20
3.2.4 附加功能二 20
3.3 特性成果 21
第四部分　总结 22
4.1 可扩展之处 22
4.2 心得体会 22
第五部分　参考文献 23

摘要

OpenFR 是一款基于 FPGA 的众脸识别系统，旨在于边缘端实现高效、低延迟的人脸检测功能。系统以 Ti60F225 开发板为核心计算平台，搭配树莓派 v2 相机采集实时图像数据，并通过 HDMI 接口将处理后的结果显示至外部显示器上。其利用深度学习模型 YOLO 对输入图像进行实时目标检测，将人脸以方框标出，实现对多个人体面部的快速识别与标注。

相比传统的 CPU 或 GPU 推理方式，OpenFR 通过将 YOLO 模型移植并优化至 FPGA 硬件中，大幅提升了识别速度与能效比。此外，系统采用模块化设计，包括图像采集模块、FPGA 加速模块、控制模块与显示模块等多个模块，各模块通过高速总线实现数据交互与同步控制。

本系统具有低功耗、高性能、可移植等优点，适用于对实时性和能效有一定要求的边缘计算场景，如智能门禁、会议签到系统及公共安全监控等。项目的开发过程涵盖了模型训练、硬件加速设计、外设驱动开发以及图像显示等关键环节。

性能测试的结果表明，OpenFR 在 540 × 540 分辨率下可实现每秒 16 帧的实时识别，平均人脸检测延迟约为 61ms，展现了 FPGA 在视觉识别任务中的显著潜力与应用价值。

第一部分　作品概述

1.1 功能与特性

OpenFR 系统基于 Ti60F225 开发板构建，前端使用树莓派 2 相机模块实时采集图像，经由 FPGA 内部的图像处理与目标检测模块识别出人脸区域，并将结果输出至 HDMI 外接显示器上进行可视化展示。系统能够实现多目标人脸识别，并通过矩形框实时标注检测结果，如下图所示。

主要功能包括：

（1）**实时视频采集**：支持 720p 分辨率的视频输入。

（2）**多人人脸检测**：基于量化 YOLO 模型实现并行检测。

（3）**可视化输出**：在 HDMI 显示器上显示标记后的图像。

（4）**低延迟处理**：FPGA 硬件级加速，处理延迟低于 70ms。

系统整体具有高效、稳定、低功耗的特性，适合嵌入式视觉识别场景部署。

1.2 应用领域

OpenFR 适用于多种需要实时人脸识别的智能化场景，特别适合在资源受限的边缘设备上部署。

典型应用包括：

（1）**智能门禁系统**：实现身份识别与进出管控。

（2）**会议签到与考勤管理**：实时识别参会人员。

（3）**智能监控系统**：在公共场所或企业环境中进行安全监测。

(4) 服务机器人视觉模块：辅助机器人实现人脸感知与交互。

由于系统是基于 FPGA 来实现核心识别逻辑，无需高功耗 CPU 支持，故可在无人值守、低能耗环境中长期运行，具有较强的应用推广价值。

1.3 主要技术特点

(1) 硬件加速架构：采用 FPGA 并行计算特性，将 YOLO 模型推理过程映射为硬件电路，提高运算吞吐量。

(2) 模型轻量化与量化处理：通过网络剪枝与 INT8 量化技术，降低模型参数量与资源占用。

(3) 高效视频流处理：内置 DMA 图像缓存机制，实现连续帧数据的无缝传输。

(4) 多模块协同设计：集成图像采集、预处理、推理、显示模块。

(5) 灵活可扩展性：系统可兼容其他目标检测模型，支持模型替换与参数调整。

(6) 低功耗运行：利用 FPGA 特性，在保证帧率的同时将能耗控制在 5W 以下。

本作品充分利用 FPGA 的并行性与可重构特性，实现了深度学习模型在嵌入式端的高效部署。

1.4 主要性能指标

指标	参数值	说明
分辨率	540*540	
帧率	16FPS	
延迟	61ms	单帧处理延迟
准确率	80%以上	标准光照下多人脸检测准确率

1.5 主要创新点

(1) 将 YOLO 模型部署到高效的 FPGA 硬件上，实现端侧实时人脸识别。

(2) 采用模型量化与流水线加速设计，显著降低计算延迟。

(3) 构建可视化实时显示系统，实现检测结果的直观输出。

(4) 实现相机与显示模块的高速数据同步，提高系统整体稳定性。

1.6 设计流程

系统设计流程如下：

(1) **模型训练与量化**：在 PC 端训练 YOLO 模型并进行量化优化。

(2) **硬件平台搭建**：在 Ti60F225 上建立图像处理与推理模块。

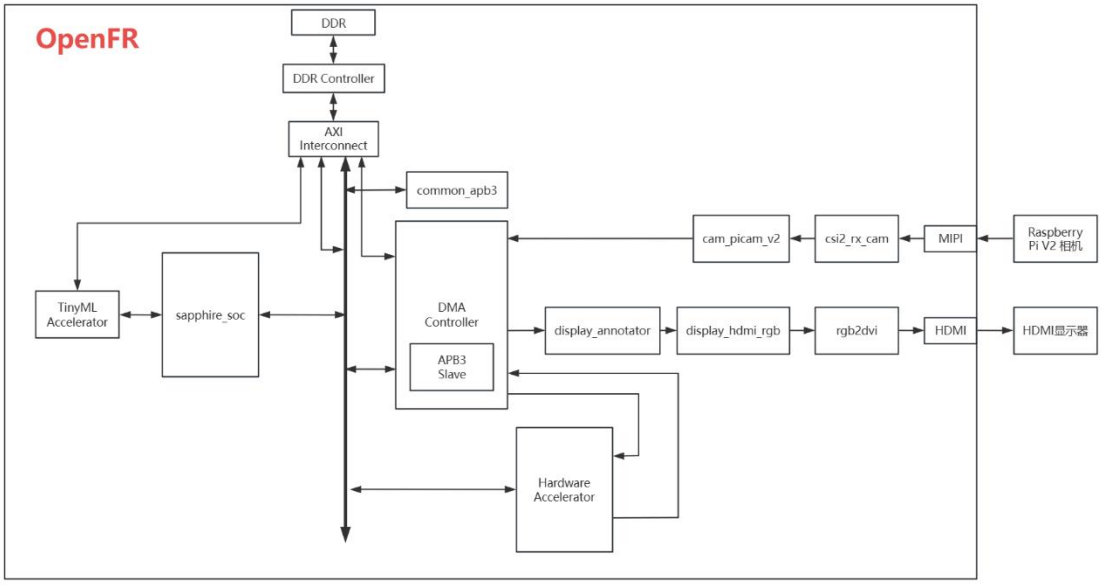
(3) **接口驱动开发**：实现摄像头采集驱动及 HDMI 显示输出模块。

(4) **系统集成与调试**：完成多模块时序同步与性能优化。

(5) **测试与验证**：通过多场景图像输入验证系统准确率与实时性。

第二部分　系统组成及功能说明

2.1 整体介绍



一、csi2_rx_cam 模块（MIPI CSI-2 接收端）

csi2_rx_cam 是用于 MIPI CSI-2 接收的模块，它负责从摄像头（树莓派 PiCam V2）采集来的 MIPI 差分信号中恢复出图像的像素数据流。

二、cam_picam_v2 模块（摄像头图像处理与 DMA 接口）

cam_picam_v2 是用于图像处理 + DMA 数据输出的模块，用于接收上一级 MIPI 解码后的数据流，将其进行必要的格式转换，并通过 DMA 写入 DDR。

三、display_annotator 模块

display_annotator 是用于视频标注和叠加的模块，主要负责在视频帧上叠加检测框。与人脸识别模块配合使用，用于在显示画面中绘制出检测到的目标矩形框。

四、display_hdmi_rgb 模块

display_hdmi_rgb 是整个视频路径的最终输出模块，负责将视频帧（带检测框的像素流）转换为 HDMI 时序信号输出到显示器。

五、rgb2dvi 模块

rgb2dvi 模块负责把来自 display_hdmi_rgb 模块的并行 RGB 视频信号（行、场同步信号+像素数据）转换成符合 HDMI/DVI 协议的高速差分信号，通过 TMDS 接口输出到显示器。

六、common_apb3 模块

common_apb3 模块通过 APB3 总线实现外设（摄像头 + 显示系统）的统一寄存器访问与控制接口。也就是说，它是 CPU 与摄像头以及显示硬件之间的控制桥梁。

七、ddr3_top 模块

ddr3_top 模块作为系统中 DDR 控制器的顶层封装模块，实现了 AXI4 总线接口 ↔ DDR3 SDRAM 接口的数据访问控制，包括写入、读取、时序校准与初始化。

它包括三个核心部分：

(1) AXI 接口层：与用户逻辑交互，接收 AXI 协议的读写请求。

(2) **控制层**：将 AXI 命令转换为 DDR3 的行列访问命令。

(3) **PHY 层接口**：直接驱动 DDR3 SDRAM 的数据线、地址线、时钟线等。

八、sapphire_soc 模块

sapphire_soc 是一个可综合的片上系统顶层模块，内部包含：CPU 核（基于 VexRiscv）、外设总线（AXI + APB）、外设控制器（UART/SPI/I2C）、系统控制逻辑（时钟/复位/中断）等。它作为 SoC 的核心逻辑，与外部 DDR 控制器、外设和调试器相连。

九、hw_accel_wrapper 模块

hw_accel_wrapper 是一个硬件加速接口模块，负责图像数据的 DMA 传输、FIFO 缓冲、硬件加速单元数据流控制与调试监控，用于 YOLO 中的加速运算数据通路。

它的作用是：从 DMA 接收图像数据，将数据送入硬件加速单元（例如神经网络计算模块、卷积模块），接收处理结果，再通过 DMA 发送回内存，同时提供一些调试信号（FIFO 溢出/下溢状态等）。

十、dma 模块

dma 是系统中数据流桥梁的核心模块。

主要功能包括：

(1) 在不同外设与内存之间搬运大块数据（如视频帧、AI 加速输入输出），减少 CPU 负担。

(2) 支持多通道数据传输。

(3) 通过 AXI4-Full 接口访问外部存储器。

(4) 通过 APB 接口与主控（SoC 控制核）进行寄存器配置与中断管理。

(5) 为摄像头输入、硬件加速结果输出、推理输入等提供独立的 DMA 通道。

十一、tinyml_top 模块

tinyml_top 是一个轻量级机器学习硬件加速单元。它实现了神经网络推理（如卷积、全连接、激活、池化等）的硬件计算加速，并通过 CPU 指令控制启动运算，通过 AXI 接口访问 DDR 存储权重和特征数据。简而言之：

它是一个可以由 CPU 控制的“AI 算法引擎”，能在硬件上快速执行模型计算。

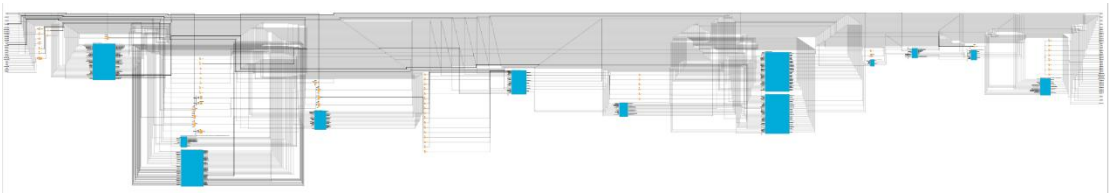
2.2 硬件系统介绍

2.2.1 硬件整体介绍:

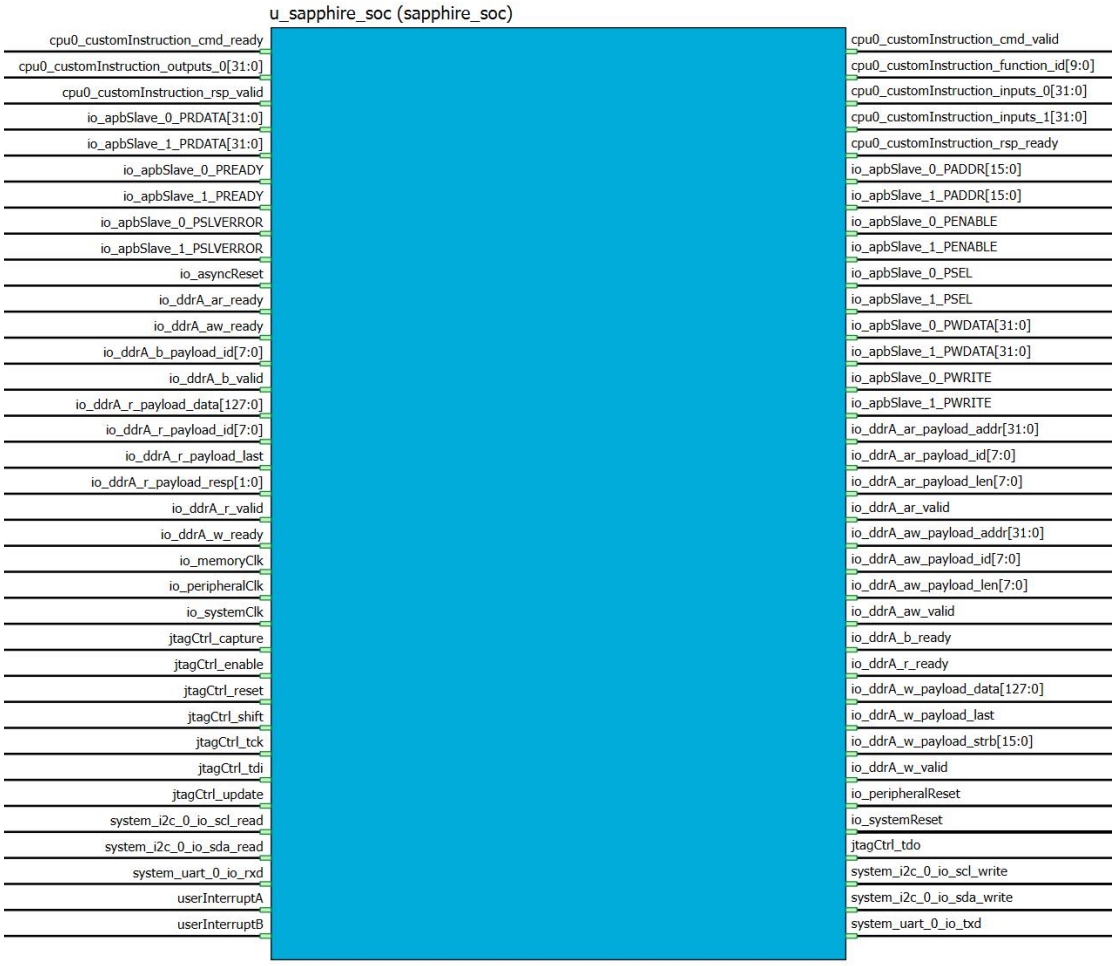
本硬件系统围绕“图像采集 → 图像预处理/缩放 → 存储/搬运 → 算法计算 → 结果叠加 → 多制式显示”构建，采用片上互联 (AXI) 与多类 DMA/FIFO 实现高带宽数据面，APB/寄存器实现低速控制面，同时集成 DDR3 存储子系统与在片调试/观测模块，形成可扩展、可维护的视觉加速平台。

2.2.2 电路各模块介绍;

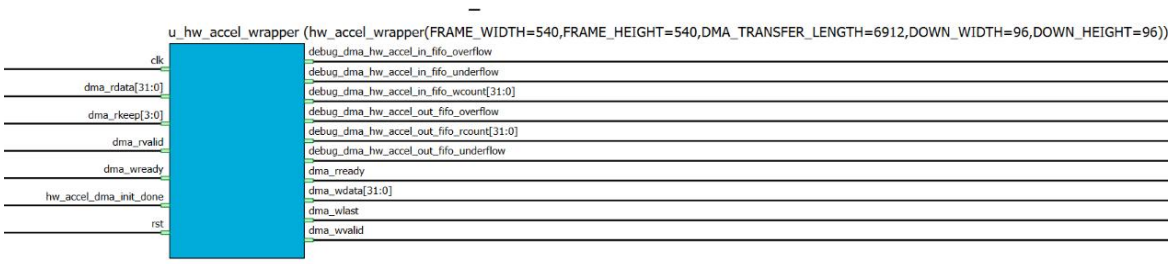
RTL 原理图



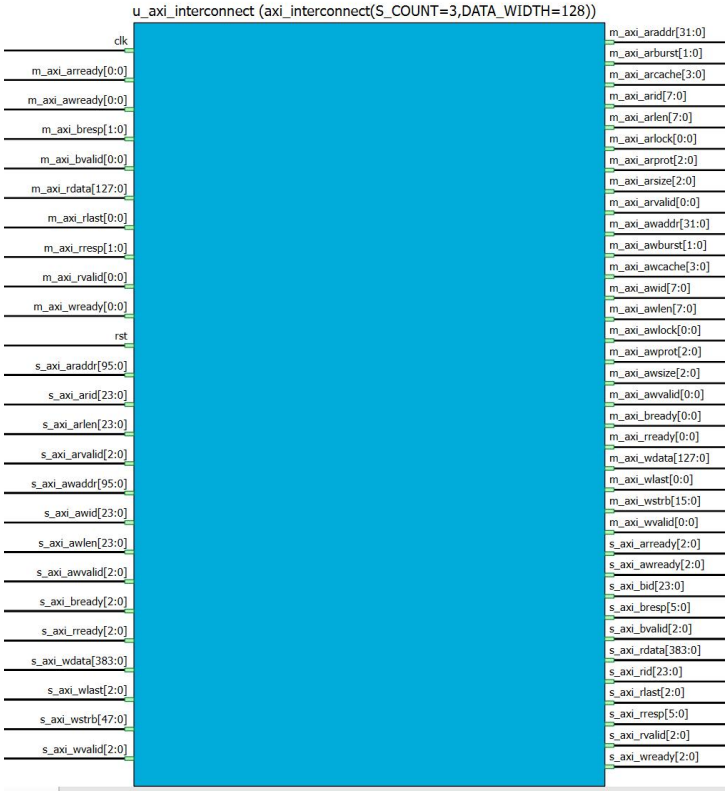
Sapphire_soc 模块



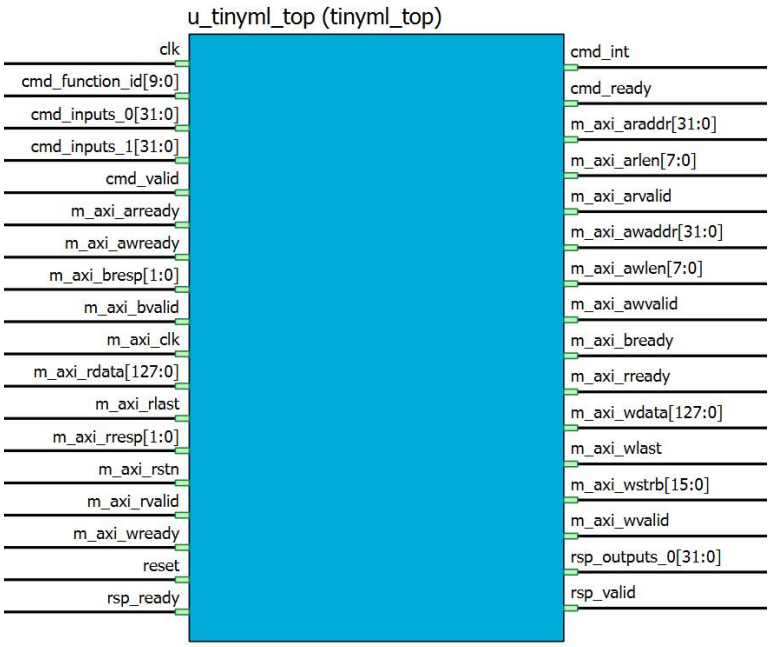
硬件加速模块



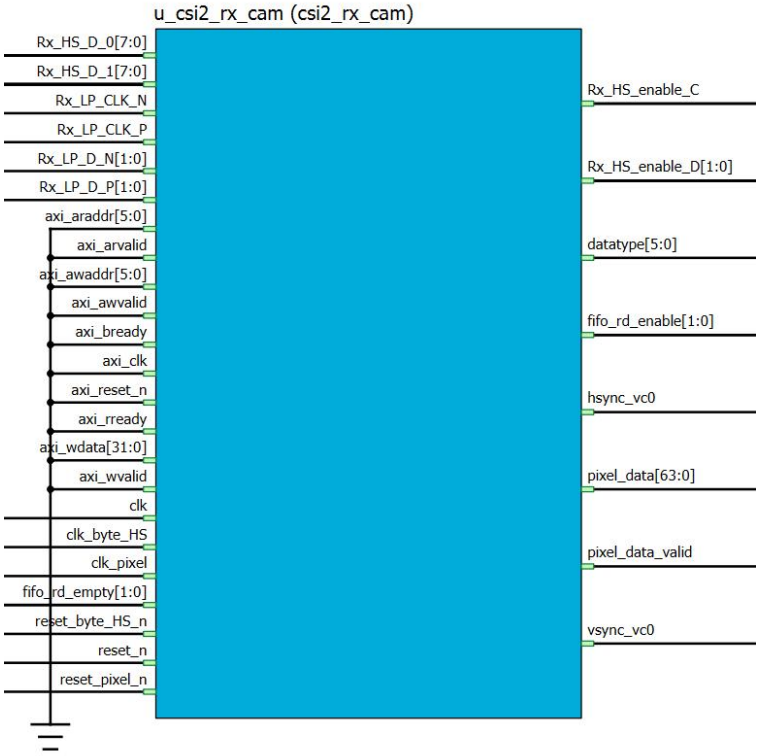
AXI 总线模块

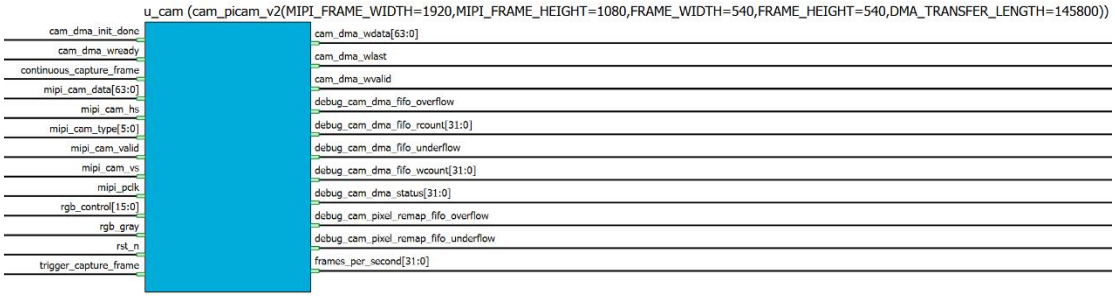


tiny_ml 模块

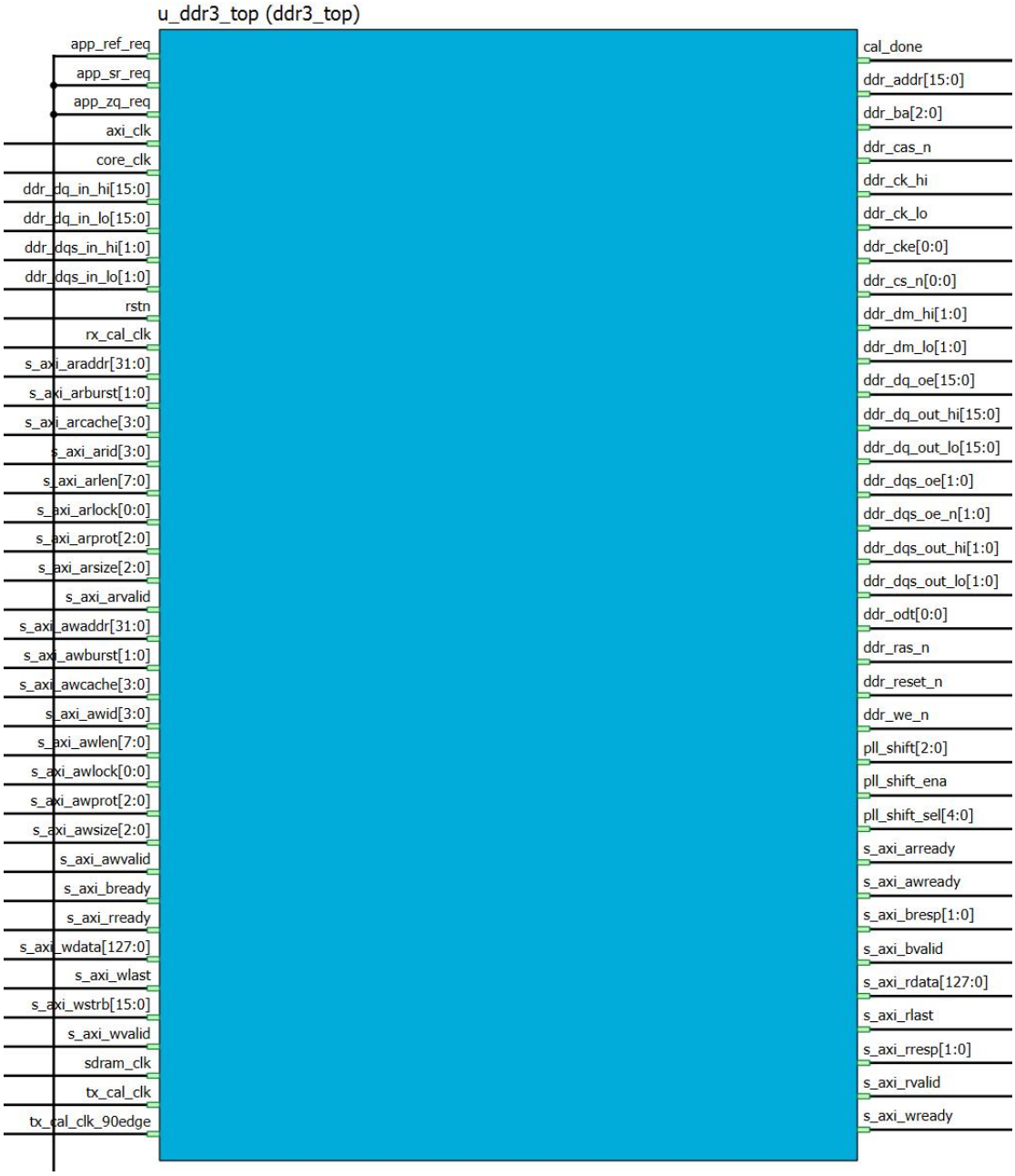


相机模块

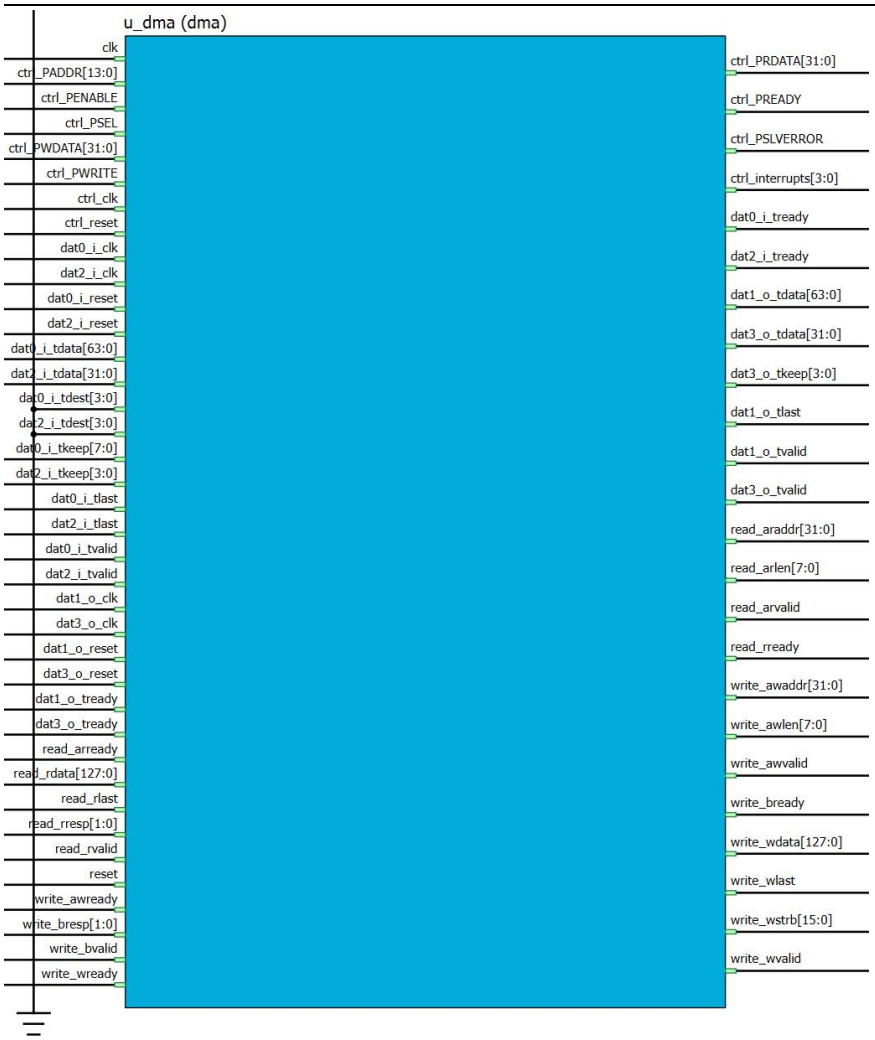

u_csi2_rx_cam (csi2_rx_cam)
Rx_HS_D_0[7:0]
Rx_HS_D_1[7:0]
Rx_LP_CLK_N
Rx_LP_CLK_P
Rx_LP_D_N[1:0]
Rx_LP_D_P[1:0]
axi_araddr[5:0]
axi_arvalid
axi_awaddr[5:0]
axi_awvalid
axi_bready
axi_clk
axi_reset_n
axi_rready
axi_wdata[31:0]
axi_wvalid
clk
clk_byte_HS
clk_pixel
fifo_rd_empty[1:0]
reset_byte_HS_n
reset_n
reset_pixel_n
Rx_HS_enable_C
Rx_HS_enable_D[1:0]
datatype[5:0]
fifo_rd_enable[1:0]
hsync_vc0
pixel_data[63:0]
pixel_data_valid
vsync_vc0


u_cam (cam_picam_v2(MIPI_FRAME_WIDTH=1920,MIPI_FRAME_HEIGHT=1080,FRAME_WIDTH=540,FRAME_HEIGHT=540,DMA_TRANSFER_LENGTH=145800))
cam_dma_init_done
cam_dma_wready
continuous_capture_frame
mipi_cam_data[63:0]
mipi_cam_hs
mipi_cam_type[5:0]
mipi_cam_valid
mipi_cam_vs
mipi_pclk
rgb_control[15:0]
rgb_gray
rst_n
trigger_capture_frame
cam_dma_wdata[63:0]
cam_dma_wlast
cam_dma_wvalid
debug_cam_dma_fifo_overflow
debug_cam_dma_fifo_rcount[31:0]
debug_cam_dma_fifo_underflow
debug_cam_dma_fifo_wcount[31:0]
debug_cam_dma_status[31:0]
debug_cam_pixel_remap_fifo_overflow
debug_cam_pixel_remap_fifo_underflow
frames_per_second[31:0]

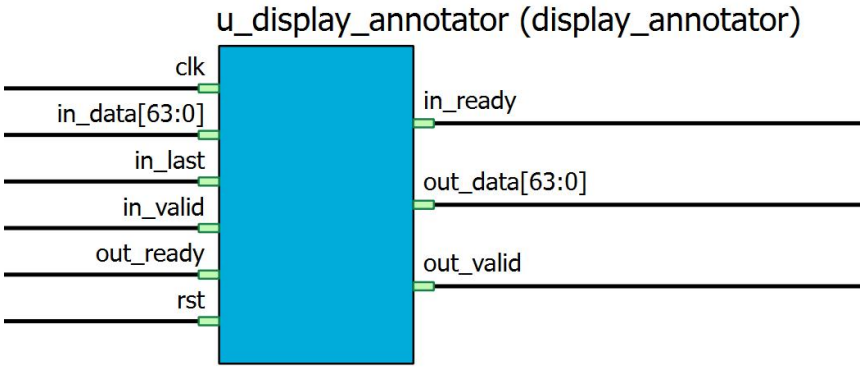
DDR 模块

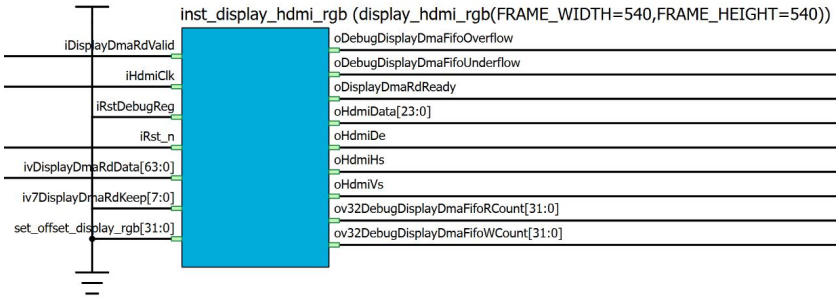

u_ddr3_top (ddr3_top)
app_ref_req
app_sr_req
app_zq_req
axi_clk
core_clk
ddr_dq_in_hi[15:0]
ddr_dq_in_lo[15:0]
ddr_dqs_in_hi[1:0]
ddr_dqs_in_lo[1:0]
rstn
rx_cal_clk
s_axi_araddr[31:0]
s_axi_arburst[1:0]
s_axi_arcache[3:0]
s_axi_arid[3:0]
s_axi_arlen[7:0]
s_axi_arlock[0:0]
s_axi_arprot[2:0]
s_axi_arsize[2:0]
s_axi_arvalid
s_axi_awaddr[31:0]
s_axi_awburst[1:0]
s_axi_awcache[3:0]
s_axi_awid[3:0]
s_axi_awlen[7:0]
s_axi_awlock[0:0]
s_axi_awprot[2:0]
s_axi_awsize[2:0]
s_axi_awvalid
s_axi_bready
s_axi_rready
s_axi_wdata[127:0]
s_axi_wlast
s_axi_wstrb[15:0]
s_axi_wvalid
sdram_clk
tx_cal_clk
tx_cal_clk_90edge
cal_done
ddr_addr[15:0]
ddr_ba[2:0]
ddr_cas_n
ddr_ck_hi
ddr_ck_lo
ddr_cke[0:0]
ddr_cs_n[0:0]
ddr_dm_hi[1:0]
ddr_dm_lo[1:0]
ddr_dq_oe[15:0]
ddr_dq_out_hi[15:0]
ddr_dq_out_lo[15:0]
ddr_dqs_oe[1:0]
ddr_dqs_oe_n[1:0]
ddr_dqs_out_hi[1:0]
ddr_dqs_out_lo[1:0]
ddr_odt[0:0]
ddr_ras_n
ddr_reset_n
ddr_we_n
pll_shift[2:0]
pll_shift_ena
pll_shift_sel[4:0]
s_axi_arready
s_axi_awready
s_axi_bresp[1:0]
s_axi_bvalid
s_axi_rdata[127:0]
s_axi_rlast
s_axi_rresp[1:0]
s_axi_rvalid
s_axi_wready

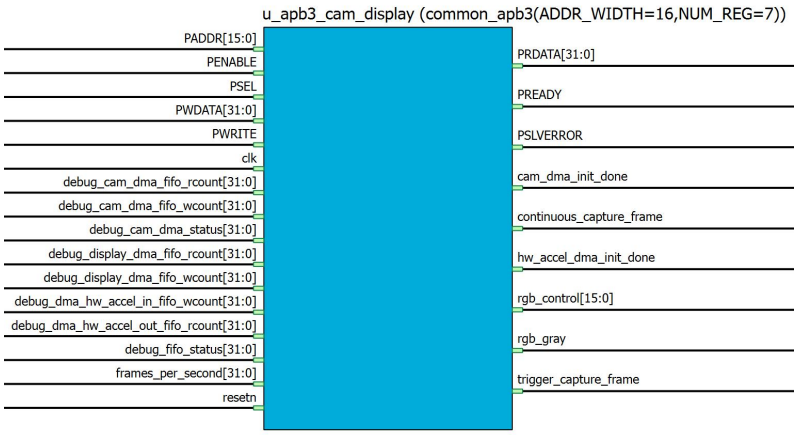
DMA 模块

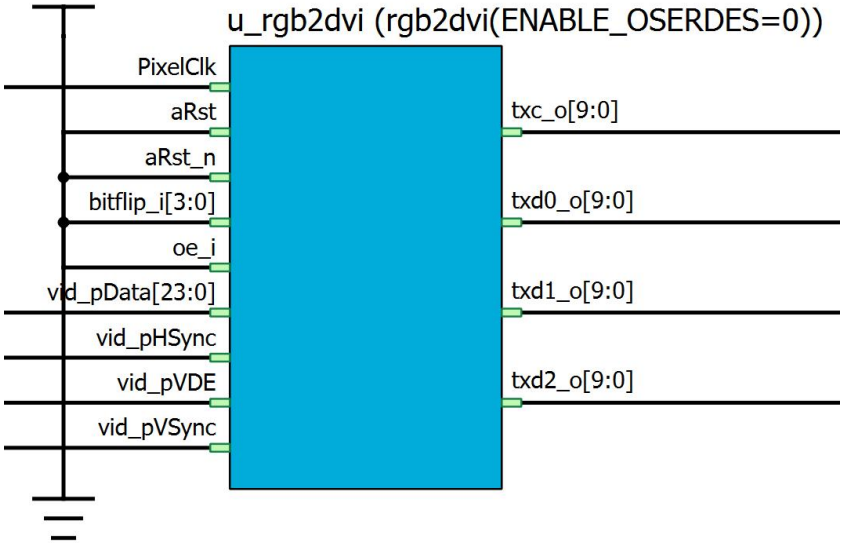

u_dma (dma)
clk
ctrl_PADDR[13:0]
ctrl_PENABLE
ctrl_PSEL
ctrl_PWDATA[31:0]
ctrl_PWRITE
ctrl_clk
ctrl_reset
dat0_i_clk
dat2_i_clk
dat0_i_reset
dat2_i_reset
dat0_i_tdata[63:0]
dat2_i_tdata[31:0]
dat0_i_tdest[3:0]
dat2_i_tdest[3:0]
dat0_i_tkeep[7:0]
dat2_i_tkeep[3:0]
dat0_i_tlast
dat2_i_tlast
dat0_i_tvalid
dat2_i_tvalid
dat1_o_clk
dat3_o_clk
dat1_o_reset
dat3_o_reset
dat1_o_tready
dat3_o_tready
read_arready
read_rdata[127:0]
read_rlast
read_rresp[1:0]
read_rvalid
reset
write_awready
write_bresp[1:0]
write_bvalid
write_wready
ctrl_PRDATA[31:0]
ctrl_PREADY
ctrl_PSLVERROR
ctrl_interrupts[3:0]
dat0_i_tready
dat2_i_tready
dat1_o_tdata[63:0]
dat3_o_tdata[31:0]
dat3_o_tkeep[3:0]
dat1_o_tlast
dat1_o_tvalid
dat3_o_tvalid
read_araddr[31:0]
read_arlen[7:0]
read_arvalid
read_rready
write_awaddr[31:0]
write_awlen[7:0]
write_awvalid
write_bready
write_wdata[127:0]
write_wlast
write_wstrb[15:0]
write_wvalid

显示模块


u_display_annotator (display_annotator)
clk
in_data[63:0]
in_last
in_valid
out_ready
rst
in_ready
out_data[63:0]
out_valid







2.3 软件系统介绍

2.3.1 软件整体介绍;

本软件系统运行在 FPGA SoC 上，完成“相机采集 → 硬件加速预处理 → 硬件加速推理 → YOLO 后处理 → 叠框显示”的端侧闭环。上位机（PC）仅

用于：串口日志查看与截图、后续更换量化模型文件与阈值调参。应用主程序为 main.cc，核心依赖包括：PiCam 相机驱动与 APB3 控制寄存器、DMA 传输封装、AXI4 硬件预处理加速器以及 YOLO 后处理模块等。

2.3.2 软件各模块介绍；

（1）主函数

职责：系统初始化、循环调度 DMA/加速器/推理/叠框。

1.初始化 init()

输入：无（上电执行）。

输出：相机、DMA、显示就绪。

流程：相机配置—>图像预处理参数—>TinyML 初始化—>DMA 初始化与优先级—>显示与 BBox 缓冲初始化—>启动相机采集。

2.主循环（推理与显示）

输入：当前显示/绘制缓冲区地址、预处理目标地址。

输出：写入 TFLM 输入张量并产生 bbox，随后更新显示。

流程：触发硬件预处理加速器—>将输入复制到 model_input —>调用 Invoke() 完成前端网络推理—>将各输出张量反量化—>调用 perform_inference() 得到候选框—> draw_boxes() 将合法框写入 bbox_array, Display 通道据此叠框。

（2）视觉采集与外设控制

职责：相机 I2C/寄存器初始化、模式切换（RGB/灰度）、触发帧采集与写入缓冲。通过 APB3 的 REG1/REG2/REG3/REG4 等寄存器完成“配置完成/开始捕获/选择通道/握手脉冲”。

输入：无（上电配置）。

输出：帧数据经 CAM S2MM DMA 写入 BUFFER_START_ADDR + 偏移的多缓冲区。

（3）DMA 传输与显示

职责：

CAM S2MM：将相机帧写入 IMAGE_START_OFFSET 开始的显存区。

HW Accel MM2S/S2MM：从显存读入、到 TinyML 输入地址写出。

DISPLAY MM2S：将图像命令+数据块与 bbox 命令+数据块送往叠加/显示模块。

以上 3 条 DMA 通道在 dma_init() 后被设定优先级并按任务触发。

输入与输出：图像数据块：IMAGE_CMD/START/END_OFFSET 描述的“命令+数据+对齐填充”的一体化块。BBox 数据块：BBOX_CMD_ADDRESS 开始的“命令+N 个 64bit 坐标+填充”。

(4) 硬件加速预处理

职责：将 540×540×RGBA 的帧做裁剪/缩放/色彩处理到 56×56×3 的 TinyML 输入，其开始/结束通过 APB3 寄存器拉高/拉低触发。预处理后的字节数由 YOLO_PICO_INPUT_BYTES 给定。

输入：540×540×RGBA 相机帧

输出：56×56×3 的数据块

(5) YOLO 后处理

职责：提供 YOLO 解码接口 perform_inference()，主程序在反量化后调用该接口。

输入：各输出层的反量化后特征、anchors、classes、阈值。

输出：筛选与抑制后的 box 列表。

(6) 叠框与显示更新

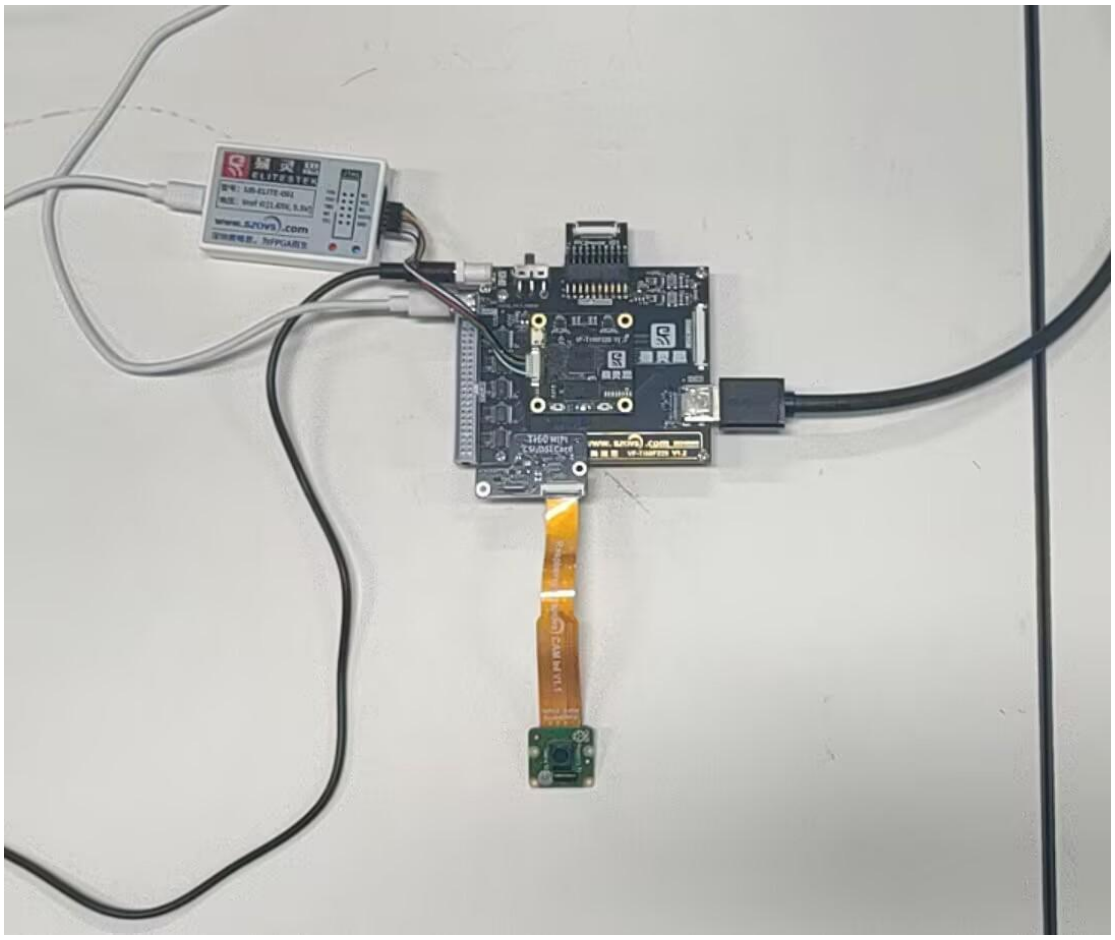
职责：draw_boxes() 将合法框打包成 64 位坐标，无效框写全 1，写完置 bbox_overlay_updated=1，Display 通道据此刷新叠框。初始化时先清空为无效框。

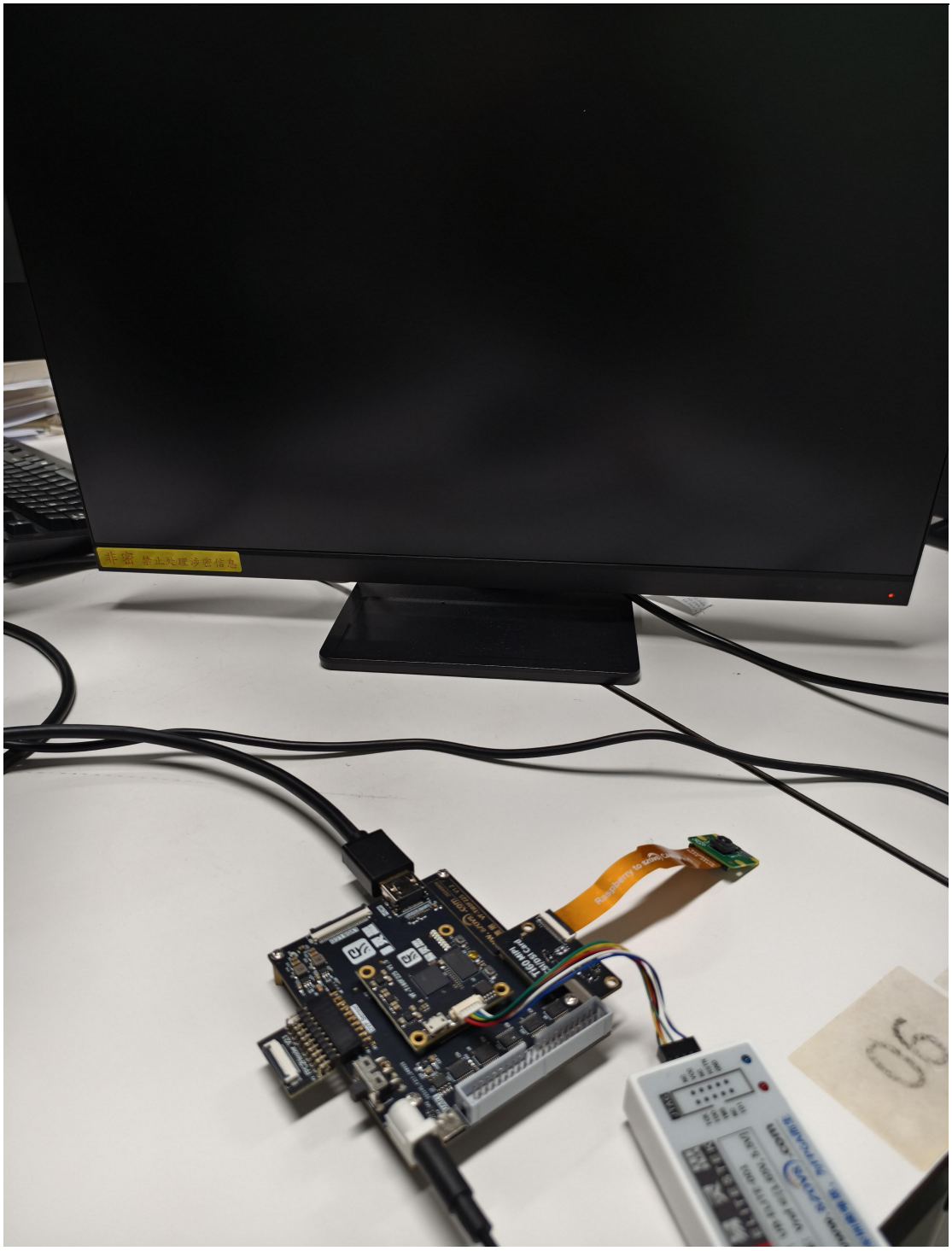
输入：人脸识别后图像。

输出：更新 bbox_array 并触发显示刷新。

第三部分　完成情况及性能参数

3.1 整体介绍

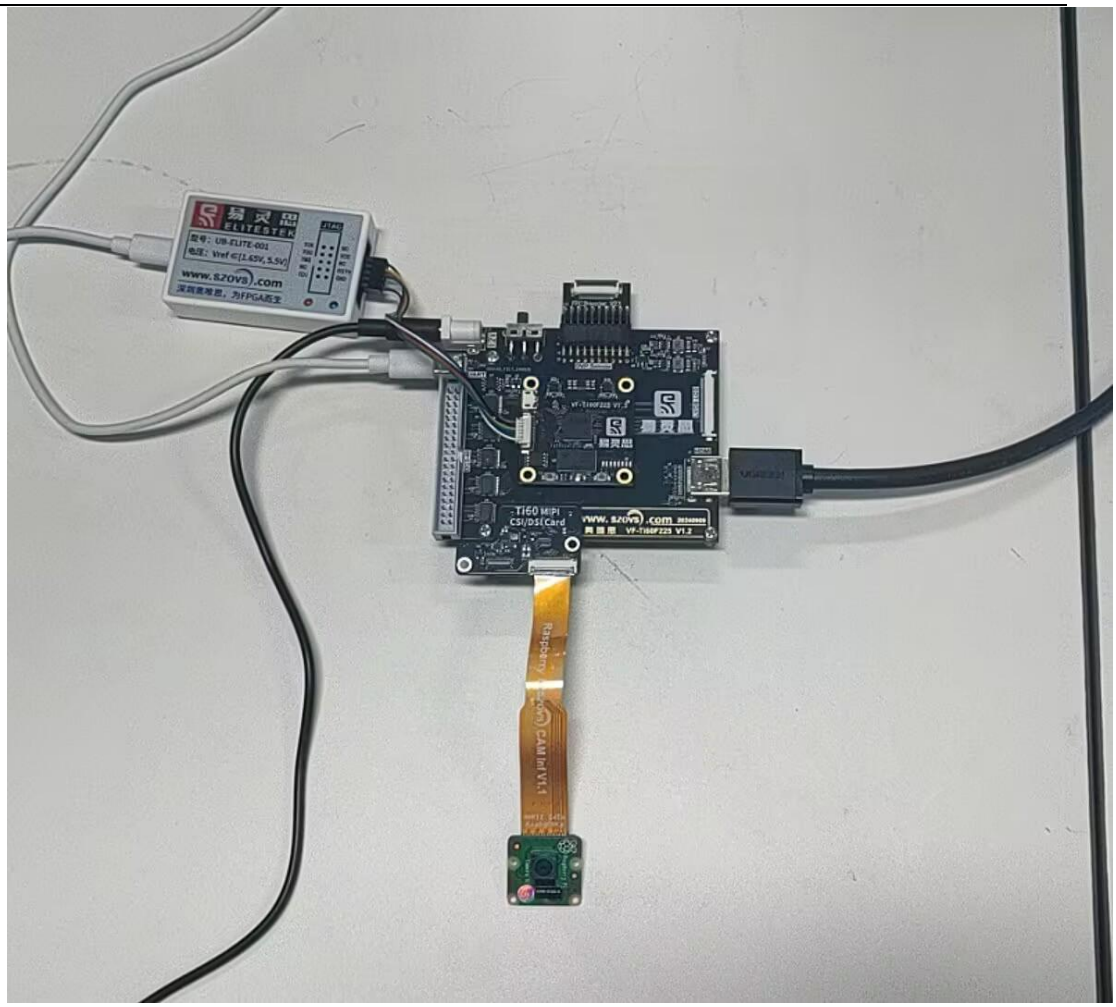




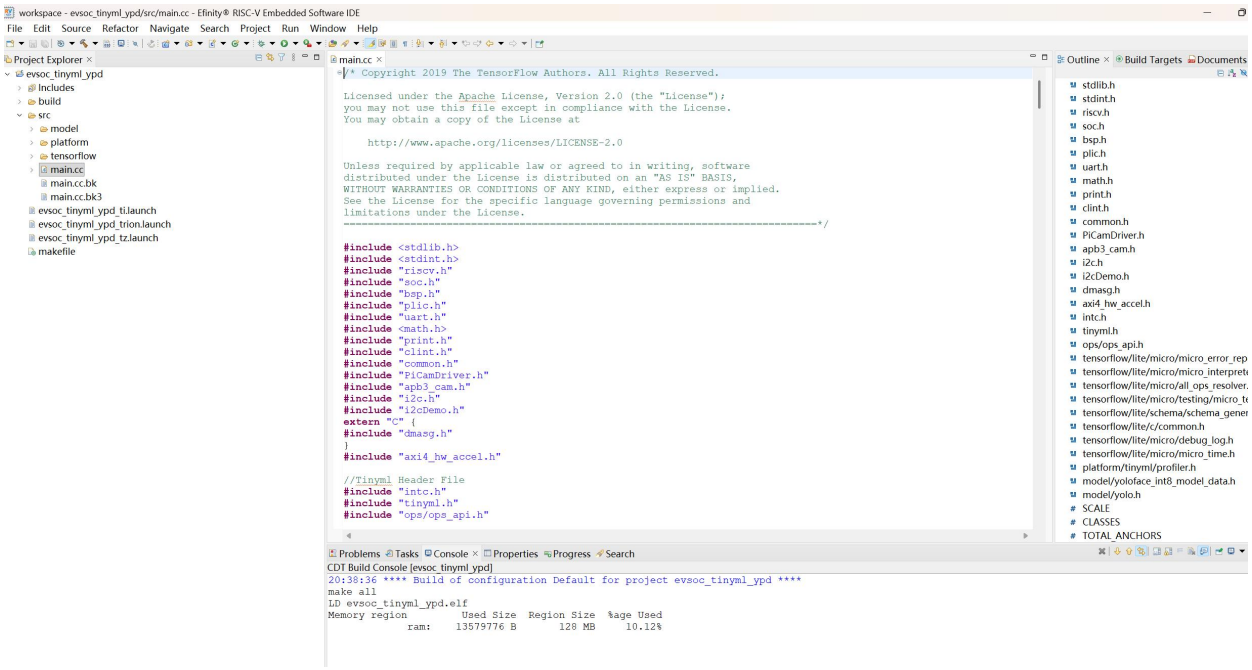
易灵思 Ti60F225 开发板，Raspberry Pi 2 相机，HDMI 显示器。Raspberry Pi 2 相机采集图像，Ti60F225 开发板对输入的图像进行人脸识别，识别后的图像输出到 HDMI 显示器。

3.2 工程成果

3.2.1 硬件成果；


易灵思
ELITESTEK
www.szovs.com
Ti60 MIPI
CSI/DSI/Card
www.szovs.com
VF-Ti60F225 V1.2
Raspberry
CAM IMX V1.1

3.2.2 软件成果；


workspace - evsoc_tinyml_ypd/src/main.cc - Efinity® RISC-V Embedded Software IDE
File Edit Source Refactor Navigate Search Project Run Window Help
Project Explorer
evsoc_tinyml_ypd
Includes
build
src
model
platform
tensorflow
main.cc
main.cc.bk
main.cc.bk3
evsoc_tinyml_ypd_ti.launch
evsoc_tinyml_ypd_trion.launch
evsoc_tinyml_ypd_tz.launch
makefile
main.cc
/* Copyright 2019 The TensorFlow Authors. All Rights Reserved.
Licensed under the Apache License, Version 2.0 (the "License");
you may not use this file except in compliance with the License.
You may obtain a copy of the License at
http://www.apache.org/licenses/LICENSE-2.0
Unless required by applicable law or agreed to in writing, software
distributed under the License is distributed on an "AS IS" BASIS,
WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied.
See the License for the specific language governing permissions and
limitations under the License.
==========*/
#include <stdlib.h>
#include <stdint.h>
#include "riscv.h"
#include "soc.h"
#include "bsp.h"
#include "plic.h"
#include "uart.h"
#include <math.h>
#include "print.h"
#include "clint.h"
#include "common.h"
#include "PiCamDriver.h"
#include "apb3_cam.h"
#include "i2c.h"
#include "i2cDemo.h"
extern "C" {
#include "dmasg.h"
}
#include "axi4_hw_accel.h"
//Tinyml Header File
#include "intc.h"
#include "tinyml.h"
#include "ops/ops_api.h"
Outline
Build Targets
Documents
stdlib.h
stdint.h
riscv.h
soc.h
bsp.h
plic.h
uart.h
math.h
print.h
clint.h
common.h
PiCamDriver.h
apb3_cam.h
i2c.h
i2cDemo.h
dmasg.h
axi4_hw_accel.h
intc.h
tinyml.h
ops/ops_api.h
tensorflow/lite/micro/micro_error_rep
tensorflow/lite/micro/micro_interprete
tensorflow/lite/micro/all_ops_resolver.
tensorflow/lite/micro/testing/micro_te
tensorflow/lite/schema/schema_gener
tensorflow/lite/c/common.h
tensorflow/lite/micro/debug_log.h
tensorflow/lite/micro/micro_time.h
platform/tinyml/profiler.h
model/yoloface_int8_model_data.h
model/yolo.h
SCALE
CLASSES
TOTAL_ANCHORS
Problems Tasks Console Properties Progress Search
CDT Build Console [evsoc_tinyml_ypd]
20:38:36 **** Build of configuration Default for project evsoc_tinyml_ypd ****
make all
LD evsoc_tinyml_ypd.elf
Memory region Used Size Region Size %age Used
ram: 13579776 B 128 MB 10.12%

3.2.3 附加功能一：

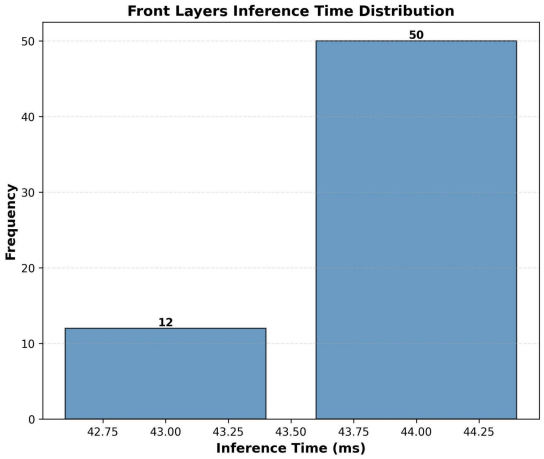
实时人数统计与警报：在检测人脸的基础上，统计画面中总人数，当超过设定阈值时弹出警告信息（左上角），适用于人群密度监控。

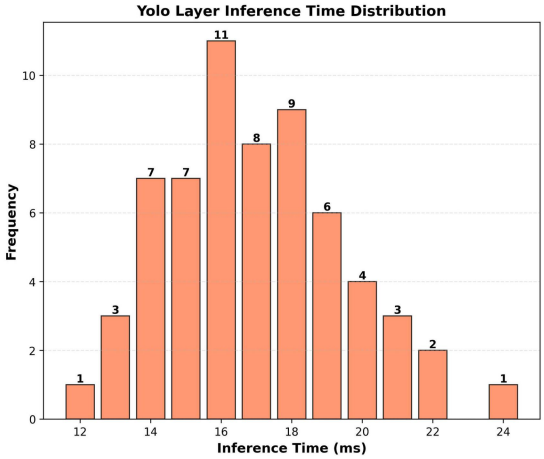
3.2.4 附加功能二：

时间段管控：设置工作时间与非工作时间的检测策略，在特定时间检测到人脸立即弹出警告（左上角），适用于安保服务。

3.3 特性成果;

指标	参数值	说明
分辨率	540*540	
帧率	16FPS	
延迟	61ms	单帧处理延迟
准确率	80%以上	标准光照下多人脸检测准确率





第四部分　总结

4.1 可扩展之处

（1）**设备联动控制**：与门禁、照明等设备联动，识别到授权人员自动开门，检测到人员自动亮灯，适用于智能化管理。

（2）**离线记录存储**：将检测到的人脸图片和时间戳存入外存，网络中断后仍能保持记录，实现数据离线存储。

（3）**低光照优化**：在光线不足环境下自动调节检测参数或启动补光，实现在夜间或昏暗环境中的识别。

4.2 心得体会

在本次基于 FPGA 的众脸识别加速系统的研发与制作过程中，我们经历了从理论探索到实践落地的完整闭环，深刻体会到边缘 AI 技术在实际应用中的挑战与魅力。

首先是复杂的技术整合。众脸识别 FPGA 加速系统涉及 RISC-V 处理器、DMA 控制器、摄像头驱动、硬件加速器等多个技术模块的协同工作。每个模块都有其特定的配置要求和时序约束，如 DMA 传输需要严格的内存对齐，摄像头 I2C 配置需要精确的时序控制，模型量化需要平衡精度与性能。这种多技术的集成要求我们不仅要理解每个组件的原理，更要掌握它们之间的交互机制。

其次是资源优化的艺术。在资源受限的嵌入式环境中，我们通过 INT8 量化将模型大小压缩至可接受范围，并降低算法运行时消耗的资源，以此来在有限资源中最大化模型性能。

然后是实时输出的保障。我们通过将 YOLO 人脸识别中最为耗时的卷积、全连接、池化等部分移植到 FPGA 上，利用 FPGA 的低延时和高并发特性，加速人脸识别过程，降低时延。

最后是不断的修改验证。由于系统复杂度高，我们借助了 AI 分析、IDE

debug 与串口打印等方法，不断排查硬件与软件问题，最终实现了利用 FPGA 加速的 YOLO 众脸识别系统。

通过这次实践，我们不仅成功实现了基于 RISC-V 和 TinyML 的众脸识别系统，而且积累了将 AI 算法落地到嵌入式平台的经验，这对于我们以后从事相关项目制作有极大帮助。

第五部分　参考文献

［1］张立国，孟子杰，金梅. 针对嵌入式设备的 YOLO 目标检测算法改进方法［J］. 高技术通讯，2024，34(4)：356 – 365.

［2］Yao Z，Douglas W，O’Keeffe S，Villing R. Faster YOLO-based object detection for edge devices［C］. Lecture Notes in Computer Science. Springer，2022：226 – 237.